

Supplementary Information

Solutions to the challenges proposed by 0xPARC

Alberto Hernandez-Espinosa

15 September 2026

Contents

S1	Questions, conventions, and contribution	2
S2	Q1: recovering an integer list	2
S3	Q2: majority using only three-input majority	2
S4	Index deconvolution: representation, recovery and controls	3
S4.1	Exact index sets and schema expansion	3
S4.2	Recovery of functional inputs	3
S4.3	Majority checks and negative controls	4
S4.4	The five-input network and pattern figures	4
S5	Q3: depth-three packed Fourier circuits	5
S5.1	A complete factorisation	5
S5.2	Charged diagonals and rotation reuse	6
S5.3	Search family and comparison	6
S6	Q4: modular power sums	6
S7	Q5 and Q6: range and exclusion constraints	7
S8	Q7: a 64-bit factorisation	7
S9	Q8: a 4096-bit factorisation	7
S10	Verification record and reproducibility	8
S10.1	Specified validation inputs	8
S10.2	Why the omitted-bound controls are informative	9
S10.3	Constraint counts	9
S10.4	Reproduction and scope of the evidence	10
S11	General capacities and developmental scope of index deconvolution	11
S11.1	The capacity map	11
S11.2	Compression and the Kolmogorov perspective	11
S11.3	Prediction, explanation and scale	13

S1 Questions, conventions, and contribution

We number the source’s puzzles Q1–Q8 in order: list recovery, majority, Fourier, modular power sums, range, exclusion of one, 64-bit factorisation, and 4096-bit factorisation [1]. The Fourier text gives 32,768 complex slots but 65,536 in its rotation example; we address both. The prime in Q4 is $q = 2^{127} - 1$. The distinct constraint field in Q5–Q8 is $\mathbb{F}_P$, with

$$P = 21888242871839275222246405745257275088548364400416034343698204186575808495617.$$

External field values must be canonical integers in $[0, P - 1]$.

This supplement states the exact conventions and gives the full arguments behind the main text. The Boolean reconstruction is described in Section S4; the finite verification record is in Section S10.

S2 Q1: recovering an integer list

Let $v = (v_0, \dots, v_{n-1})$ be positive integers and let an oracle return exact dot products. With no bound, first query $(1, \dots, 1)$ to obtain $S = \sum_i v_i$. For $n > 1$, put $B = S + 1$ and query $(1, B, \dots, B^{n-1})$, obtaining

$$R = \sum_{i=0}^{n-1} v_i B^i.$$

Since $0 < v_i < B$, repeated division by B recovers the digits without carries between positions. A known bound $v_i \leq U$ permits $B = U + 1$ in advance and one query. For $n = 1$, the first query alone recovers the list.

Proposition 1. *For $n \geq 2$ with unbounded positive integers and rational query coefficients, two queries suffice and one query cannot always suffice.*

Proof. Sufficiency follows from the adaptive construction above. For any fixed rational query vector x , clearing denominators leaves an integer linear map from $\mathbb{Z}^n$ to $\mathbb{Z}$. Its rational nullspace has a nonzero integer vector z . Choose positive v_i sufficiently large that $v_i + z_i > 0$. Then v and $v + z$ are different positive lists with the same answer. $\square$

An idealized one-query solution uses coefficients linearly independent over $\mathbb{Q}$. For example, $1, t, \dots, t^{n-1}$ for a transcendental real t are independent: equality of two answers would give a nonzero integer polynomial vanishing at t . This is an injectivity argument in an exact-real model, not a floating-point recovery algorithm. A bounded integer encoding already needs roughly $n \log_2 B$ answer bits; reducing the number of queries does not remove that information requirement.

S3 Q2: majority using only three-input majority

Let f be a monotone Boolean function and choose distinct active variables a, b, c . Write $f_{b \leftarrow a}$ for the function formed by substituting a for b , and similarly for the other identifications. Then

$$f = \text{MAJ}_3(f_{b \leftarrow a}, f_{c \leftarrow b}, f_{a \leftarrow c}).$$

If the three bits agree, every substitution leaves them unchanged. Otherwise one substitution leaves the assignment unchanged, one increases a bit, and one decreases a bit. By monotonicity the three outputs are f , a value at least f , and a value at most f . Their majority is f .

Odd-input majority is monotone and self-dual: complementing all input bits complements its output. Identification preserves both properties. A monotone self-dual Boolean function on at most two active variables is a projection: its values at 00, 11 are 0, 1, and the values at 01, 10 are opposite. Thus recursion terminates in input wires and uses only MAJ₃ gates.

The construction represents subproblems by a positive odd total of weights over the original coordinates. Identification merges weights, a weight greater than half the total immediately returns its input, and otherwise the first three active variables are used. Memoisation identifies equal weight tuples. Every operand is an external input or an earlier gate; ordered duplicate ports and free fan-out are permitted. Constants and inversions are absent.

With $T(2) = 0$ and $T(k) \leq 1 + 3T(k-1)$, a loose unshared bound is $T(n) \leq (3^{n-2} - 1)/2$ for $n \geq 3$. This proves the requested existence for 2025 inputs but is not a compactness guarantee. No 2025-input circuit is materialised here. Gate counts below refer only to emitted circuits. Exhaustive checks cover odd sizes through 11; exact decision diagrams cover 13 and 15. The symbolic target is built independently by a threshold recurrence.

For comparison, a naive majority of disjoint group majorities fails: the groups 110, 110, 000 produce local votes 1, 1, 0 and hence output one, although only four of the nine original inputs are one.

S4 Index deconvolution: representation, recovery and controls

S4.1 Exact index sets and schema expansion

Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ be a deterministic output column. States are ordered by $\iota(x) = \sum_{i=0}^{n-1} 2^i x_i$, with bit zero varying fastest. The active index set $\mathcal{I}_f$ consists of all indices with output one. Membership in this set specifies the complete output column.

For a schema $\sigma \in \{0, 1, *\}^n$, let $J_1(\sigma)$ be its fixed-one positions and $J_*(\sigma)$ its don't-care positions. Define the decimal anchor and the offset family by

$$a_\sigma = \sum_{i \in J_1(\sigma)} 2^i, \quad \Omega_\sigma = \left\{ \sum_{i \in J_*(\sigma)} 2^i z_i : z_i \in \{0, 1\} \right\}.$$

The sumandos are the fillings of the schema's own don't-care positions; Ω_σ is the numerical offset family these fillings generate. The indices represented by the schema are exactly $a_\sigma + \Omega_\sigma$. Fixed-one and free positions are disjoint, so adding their encodings cannot create a carry. A covering family $\mathcal{C}$ represents the output precisely when

$$\mathcal{I}_f = \bigcup_{\sigma \in \mathcal{C}} (a_\sigma + \Omega_\sigma).$$

This is a union, since schemata may overlap. One must retain each anchor's associated offset family; arbitrary cross-pairing of anchors and offsets need not preserve the function. A cover by complete assignments always exists, but a compact cover or a minimal cover is not guaranteed.

For three-input majority, $\mathcal{C} = \{11*, 1*1, *11\}$ gives anchor/offset pairs $(3, \{0, 4\})$, $(5, \{0, 2\})$, $(6, \{0, 1\})$. The expansion is $\{3, 5, 6, 7\}$. All three inputs affect the function, although each schema has a free position. Disconnected coordinates are free in every schema; freedom in one schema does not imply disconnection.

S4.2 Recovery of functional inputs

From the complete column, recover

$$I_c(f) = \{i : \exists x, f(x) \neq f(x \oplus e_i)\},$$

where $x \oplus e_i$ flips bit i . These are the functional inputs. If $i \notin I_c(f)$, changing only bit i never changes the output. Any two states agreeing on $I_c(f)$ can be joined by a sequence of flips outside $I_c(f)$, so their outputs agree. Therefore there is a well-defined reduced function g with $f(x) = g(x|_{I_c(f)})$. Its truth table is obtained by restricting the full column and checking consistency across all free assignments. Expanding it to the original coordinates reconstructs f exactly. This proves functional recovery from a complete deterministic repertoire. Gate identification can match g to a declared family; multiple names or parameterisations may describe the same function.

Testing all coordinate pairs costs at most $n2^{n-1}$ comparisons per output column, with $O(2^n)$ storage when the full column is retained. The method does not remove the exponential cost of supplying an arbitrary complete repertoire. A compact schema family is a representation benefit, not a universal guarantee that this inverse computation is inexpensive.

For a network with observed outputs for every input state, apply the same argument to each output column to recover a functionally equivalent map. Structurally declared but functionally inactive inputs are not identifiable. If only a circuit's final output is observed, internal gates and their wiring are not identifiable either. Partial or noisy observations do not satisfy this exact-recovery premise.

S4.3 Majority checks and negative controls

For odd n , setting exactly $(n-1)/2$ of the other bits to one shows that every input of majority is functional. Complementation gives exactly 2^{n-1} active indices. The added manuscript check evaluates the emitted circuits for $n = 1, 3, 5, 7, 9, 11$, applies functional-input detection and column reduction, and compares the result with the independent condition $\sum_i x_i > n/2$ on every state. This covers $2 + 8 + 32 + 128 + 512 + 2048 = 2730$ states. The check also expands the three schemata above exactly.

A free-input control embeds three-input majority in five coordinates. All 32 states recover $I_c = \{0, 1, 2\}$ and the eight-entry majority table; coordinates 3 and 4 do not affect the output. An incorrect-circuit control uses a majority of three disjoint three-input majorities. It has nine functional inputs and 256 active indices, yet its full output column differs from nine-input majority. Thus neither the number of functional inputs nor the size of the active set establishes equality; exact membership is checked. The main text's 110,110,000 assignment is one explicit disagreement.

S4.4 The five-input network and pattern figures

The worked network uses the emitted five-input circuit, with

$$\begin{aligned} g_0 &= \text{MAJ}_3(x_0, x_2, x_4), & g_1 &= \text{MAJ}_3(x_0, g_0, x_3), \\ g_2 &= \text{MAJ}_3(x_1, x_3, x_4), & g_3 &= \text{MAJ}_3(x_0, x_1, g_2), \\ g_4 &= \text{MAJ}_3(x_2, x_3, x_4), & g_5 &= \text{MAJ}_3(x_1, x_2, g_4), \\ y &= g_6 = \text{MAJ}_3(g_1, g_3, g_5). \end{aligned}$$

The figures show this circuit's connections, not a generic illustrative network. All seven internal outputs are evaluated for all 32 inputs. The output column is compared state by state with $\sum_i x_i \geq 3$; functional-input detection recovers all five coordinates.

To generate the schema family, choose every three-element subset of the five input positions, fix those bits to one and leave the other two free. This gives $\binom{5}{3} = 10$ schemata. Expanding their associated anchors and offsets and taking the union gives precisely the circuit's 16 active indices.

The displayed 111 ** schema has anchor 7 and offsets 0, 8, 16, 24. Its four complete input states, together with their internal outputs, are

Index	Input $x_0 \dots x_4$	Internal outputs $g_0 \dots g_6$
7	11100	1101011
15	11110	1111111
23	11101	1111111
31	11111	1111111

For each input, the figure-generation check retains a pair of states that differ only at that input and have different outputs. The displayed pair is index 6 (01100, output zero) and index 7 (11100, output one). The complete repertoire, internal signals, schemata and checks accompany the figures as numerical data. Both diagram topology and table entries are derived from the same circuit, and every schema is checked for exact membership before rendering. These controls establish the stated finite reconstruction behaviour and illustrate its limitations; they are not a new synthesis algorithm or a claim to recover hidden internal topology.

S5 Q3: depth-three packed Fourier circuits

We use the forward unnormalised DFT and rotation convention

$$(Fx)_r = \sum_{j=0}^{N-1} x_j e^{-2\pi i r j / N}, \quad R_k(x)[r] = x[(r + k) \bmod N].$$

The primitives are vector addition, rotation by any offset, and componentwise multiplication by a known complex vector. Their charged costs are respectively 4, 6100, and 5600 microseconds. Every known-vector multiplication consumes one multiplication level, including masks. This explicitly states how constants are charged under the puzzle's simplified model.

S5.1 A complete factorisation

At DIF stage $s = 0, \dots, L - 1$, where $N = 2^L$, take blocks of length $m = N/2^s$ and $h = m/2$. For each block and $0 \leq t < h$, the stage maps

$$y_t = x_t + x_{t+h}, \quad y_{t+h} = (x_t - x_{t+h})e^{-2\pi i t / m}.$$

The first output half contains the even-indexed frequencies after recursively transforming its sum inputs; the second contains the odd-indexed frequencies after transforming its twiddled differences. Induction on L gives the DFT in bit-reversed frequency order. Applying the bit-reversal permutation to the output therefore yields the natural-order DFT.

Partition the L stages into at most three nonempty contiguous intervals. Each interval is fused into a linear transform; the final transform includes the bit-reversal permutation. For interval $[a, b)$, let $m = N/2^a$ and $h = N/2^b$. Before final permutation, each row r has precisely these input columns:

$$\lfloor r/m \rfloor m + (r \bmod h) + ht, \quad 0 \leq t < 2^{b-a}.$$

Following a row backwards through the butterfly stages gives one path to each such column. An upper output contributes coefficients $+1, +1$; a lower output contributes the stage twiddle times $+1, -1$. Products along the chosen path determine the coefficient exactly. In the last transform replace r inside this rule by its L -bit reversal. This gives explicit, lazily generated coefficient vectors, without a dense full-size matrix.

S5.2 Charged diagonals and rotation reuse

For each fused matrix A , set $d_k[r] = A[r, (r + k) \bmod N]$. With K the set of nonzero cyclic diagonals,

$$Ax = \sum_{k \in K} d_k \odot R_k(x).$$

For a baby-step size $b_0 \mid N$, write $k = gb_0 + i$, $0 \leq i < b_0$, and regroup:

$$Ax = \sum_g R_{gb_0} \left(\sum_{i: gb_0+i \in K} R_{-gb_0}(d_{gb_0+i}) \odot R_i(x) \right).$$

This identity follows by evaluating both sides at a slot: the outer rotation cancels the opposite rotation of the known coefficient. Rotating known coefficient data is part of its offline definition; every rotation of the encrypted vector is an explicit charged node. If I and G are the used baby and giant indices, the emitted transform contains $|K|$ multiplications, $|K| - 1$ additions, and $|I \setminus \{0\}| + |G \setminus \{0\}|$ rotations. There is one multiplication level per fused transform.

For nonfinal intervals, K is the set of $ht \bmod N$ with $-(2^{b-a} - 1) \leq t \leq 2^{b-a} - 1$. For the final interval, $h = 1$, and each row contributes a cyclic interval of m offsets starting at $\lfloor \text{rev}_L(r)/m \rfloor m - r$. A cyclic interval union finds the exact support without enumerating an $N \times N$ matrix.

S5.3 Search family and comparison

All ordered contiguous partitions into one, two, or three groups are examined: $1 + (L - 1) + \binom{L-1}{2}$, giving 106 and 121. For every block we evaluate every power-of-two baby size dividing N . Costs add across blocks, so selecting the best baby size independently per block evaluates the complete Cartesian family without materializing redundant tuples. Ties use multiplication count, rotation count, partition order, then baby-size order. The ledger retains every evaluated block choice and its status. The result is best within this family, not a globally optimal arithmetic circuit.

The dense depth-one baseline uses all N diagonals with the same rotation regrouping and cost model. Its best baby sizes are 128 and 256 for the two dimensions (the symmetric 256 alternative ties at $N = 32768$). Numerical validation executes every emitted primitive, compares with an independent FFT, and requires normalised maximum error at most 10^{-10} . Every basis vector through $N = 128$ is also compared with the direct exponential formula. No ciphertext latency is inferred from the plaintext runtime.

S6 Q4: modular power sums

For the stated prime $q = 2^{127} - 1$, only the zero solution exists. The linear equation gives $d = -(a + b + c)$. Expanding the cube equation gives

$$a^3 + b^3 + c^3 + d^3 = -3(a + b)(a + c)(b + c).$$

Since $q \neq 3$ and $\mathbb{F}_q$ has no zero divisors, one of the three pair sums is zero. Relabel so $b = -a$ and hence $d = -c$. The square equation becomes $2(a^2 + c^2) = 0$. Here $q \neq 2$ and $q \equiv 3 \pmod{4}$. If $c \neq 0$, then $z = a/c$ would satisfy $z^2 = -1$. But Fermat's theorem gives $1 = z^{q-1} = (-1)^{(q-1)/2} = -1$, a contradiction. Thus $c = 0$ and then $a = b = d = 0$. This proof uses the primality stated in the puzzle.

Finite checks verify the polynomial identity over the integers and exhaustively enumerates small fields. Primes 7, 11, and 19 have only zero. Characteristic 3 is exceptional: $(1, 1, 1, 0)$ satisfies all three equations. For 5 and 13, -1 has square roots 2 and 5 respectively, giving nonzero opposite-pair solutions. These examples illustrate the assumptions; they do not substitute for the proof at q .

S7 Q5 and Q6: range and exclusion constraints

A quadratic row means $A(w)B(w) = C(w)$ in $\mathbb{F}_P$, with all three expressions linear in the signals. For a 64-bit range use auxiliary bits $b_0, \dots, b_{63}$:

$$b_i(b_i - 1) = 0 \quad (0 \leq i < 64), \quad x = \sum_{i=0}^{63} 2^i b_i.$$

Every satisfying bit is zero or one because a field has no zero divisors. The sum lies in $[0, 2^{64} - 1] \subset [0, P - 1]$, so its canonical representative equals x as an integer. Conversely the ordinary binary expansion of any such x provides a witness. A field circuit cannot distinguish x from an external integer $x + P$; canonical input validation is therefore explicit.

To exclude one, impose $(r - 1)s = 1$. It has a witness exactly when $r \neq 1$: take the inverse $s = (r - 1)^{-1}$. At $r = 1$ the equation would be $0 = 1$.

S8 Q7: a 64-bit factorisation

Range-constrain public n and private u, v to 64 bits, enforce $uv = n$, and enforce $(u - 1)s = 1$ and $(v - 1)t = 1$. The system also explicitly enforces $u, v \geq 2$ using an inverse for the sum of each factor's bits above position zero. That sum lies between 0 and 63, so its nonzero field value means the factor is at least two. These extra rows make the public-input promise robust.

Completeness follows by taking the binary expansions and inverses from every nontrivial integer factorisation. For soundness, range constraints give $0 \leq u, v < 2^{64}$, so $uv < 2^{128} < P$. Both uv and n are therefore canonical integers, and the field equality lifts to $uv = n$ over the integers. The inverse constraints exclude trivial factors. The circuit checks supplied factors; it does not find them.

S9 Q8: a 4096-bit factorisation

Put $B = 2^{64}$ and represent u, v, n by 64 limbs each: $u = \sum_i u_i B^i$, $v = \sum_i v_i B^i$, and $n = \sum_i n_i B^i$. Range-constrain every limb, including public n_i , to 64 bits. Introduce 4096 partial-product signals and 129 carries, with rows

$$\begin{aligned} u_i v_j &= q_{ij} & (0 \leq i, j < 64), \\ \sum_{i+j=k} q_{ij} + c_k &= n_k + B c_{k+1} & (0 \leq k < 128), \\ c_0 &= c_{128} = 0, \end{aligned}$$

where $n_k = 0$ for $k \geq 64$. Constrain $c_1, \dots, c_{127}$ to 70 bits. For each factor take the sum of its 4095 bits other than global bit zero and require an inverse of that sum. Since $4095 < P$, this enforces that the factor is at least two.

Completeness. Every integer factorisation $n = uv < 2^{4096}$ with $u, v \geq 2$ has factors below 2^{4096} . Use their base- B limbs, exact partial products, and ordinary schoolbook carries. Inductively $c_k \leq 64(B - 1)$: with at most 64 products, the column total is at most $64(B - 1)^2 + 64(B - 1) = 64B(B - 1)$. Hence $c_{k+1} \leq 64(B - 1) < 2^{70}$. The output fits 64 limbs, so all higher output limbs and the final carry are zero. Binary decompositions and the indicated inverses complete the witness.

Soundness. Every $u_i v_j < B^2 = 2^{128} < P$, so each partial-product field equation fixes the exact canonical product q_{ij} . The left side of any column is at most $64(B-1)^2 + (2^{70} - 1)$; the right side is at most $(B-1) + B(2^{70} - 1)$. Both are nonnegative and less than $2^{135} < P$. Thus every column equality is an integer equality, not merely a congruence. Multiplying column k by B^k and summing cancels the intermediate carries, giving $uv = n + B^{128}c_{128} - c_0 = n$. The zero upper outputs prevent overflow past the 64 public limbs, and the inverse constraints make both factors nontrivial. This proves both directions independently of witness generation.

S10 Verification record and reproducibility

The exporter writes only generic quadratic equalities and input declarations. All private and auxiliary assignments are supplied explicitly. Compilation uses Circom 2.2.3, BN128, optimisation level zero, and disabled witness sanity checks; the R1CS remains enforced. Snarkjs 0.7.6 checks valid witnesses. A separate Python evaluator reads exported compiled rows, and malicious assignments are evaluated directly against those rows. Witness-generator refusal is never counted as soundness evidence.

The audit covers range boundaries, nonbinary bits, factor zero and one, incorrect public products, partial products, carry bits and endpoints, high limbs, overflow, and modular aliases. An explicit false factorisation $2((P+7)/2) = 7 + P$ satisfies multiplication columns with unrestricted field carries; the released carry ranges reject it. Deliberately removing the relevant range rows makes this negative control pass, demonstrating why those bounds are required. Public-limb and factor-range omission controls are checked too.

S10.1 Specified validation inputs

The 13 Fourier vectors at each size are: the zero vector; the constant vector $2 - 3i$; four impulses of amplitude $1 - 2i$ at positions $0, 1, \lfloor N/3 \rfloor, N-1$; three modes $x_j = \exp(2\pi i k j / N)$ for $k = 1, N/4, N-1$; and four complex Gaussian vectors. For the latter, independent standard normal samples supply the real and imaginary parts, using the default NumPy random generator with seeds 101, 202, 303 and 404. The recorded environment uses NumPy 2.4.3 and Python 3.13. The normalised maximum error is

$$\varepsilon = \frac{\max_r |y_r - y_r^{\text{ref}}|}{\max(1, \max_r |y_r^{\text{ref}}|)},$$

where the reference is the unnormalised forward transform evaluated by an independent FFT. A finite error no larger than 10^{-10} is required. Complex arrays use double precision. Small-size checks additionally compare every basis vector through $N = 128$ against the direct exponential formula. The target runs impose a 900-second limit per vector and a 4 GiB memory ceiling. Their plaintext elapsed times are validation costs, not estimates of encrypted execution.

The valid arithmetic witnesses exercise the following inputs, with $B = 2^{64}$. In Q7 and Q8 the public value is the product of the listed factors.

Question	Valid inputs
Q5	$x = 0, 1, B - 1$
Q6	$r = 0, 2, P - 1$
Q7	$(u, v) = (2, 3), (3, 2), (65537, 65539), (2, (B - 2)/2)$
Q8	$(u, v) = (2^{2047} + 123, 2^{2047} + 321), (2, 2^{4094} + 1), (2^{4094} + 1, 2)$

Valid assignments are checked both before and after compilation. Invalid assignments are injected directly into the compiled rows, so failure of a witness-generation routine is not used as rejection

evidence. Mutations exercise nonbinary bits, out-of-range values, wrong inverses, zero and unit factors, changed public products, partial products, carry bits, endpoint carries, high factor and public limbs, and overflow beyond 4096 bits. The 64-bit range interface also rejects noncanonical external integers $-1, P, P + 1$ before they can alias field values.

S10.2 Why the omitted-bound controls are informative

For the large-factor carry control, take $u = 2$, $v = (P + 7)/2$ and public $n = 7$. The ordinary product is $P + 7$. Write the factors and n in limbs, set $c_0 = 0$, and compute successive field carries by

$$c_{k+1} = B^{-1} \left(\sum_{i+j=k} u_i v_j + c_k - n_k \right) \pmod{P}.$$

All multiplication columns are then satisfied. Summing the weighted columns shows that $c_{128} = B^{-128}(uv - n) = 0$ in the field, so even zero endpoints cannot reject the false integer equality by themselves. The released system rejects these carries because of its 70-bit range equations. Deleting those range equations makes this assignment pass, isolating their role.

The small-factor control uses the same modular alias and isolates the 64-bit factor bound. A public-limb control replaces n_0 by $n_0 + B$ and n_1 by $n_1 - 1$, leaving the represented integer unchanged but violating the canonical limb range. A separate overflow control attempts to represent $2^{4095} \cdot 2$ with all public limbs zero. These controls check distinct ways in which a field assignment could violate the intended integer statement.

Checks behind the operation-network diagrams. The query diagram computes the sum and weighted contributions of $(3, 8, 19)$, then repeatedly divides the exact answer by 31 and checks recovery of all three entries. The eight-slot Fourier diagram is generated from three explicit butterfly matrices and the bit-reversal permutation. Their product is compared with the direct exponential matrix, and the emitted packed circuit is checked on all eight basis vectors against that same formula, with a maximum-error tolerance of 10^{-12} . The target-size cost bars and stage groups are read from the recorded search results. For the decimal multiplication illustration, the successive column totals are 21, 28, 10, 1, with output digits 1, 8, 0, 1 and outgoing carries 2, 2, 1, 0. The check verifies that their weighted digit sum equals $23 \cdot 47 = 1081$. These small examples explain the operations; the large-dimension numerical and compiled checks remain those reported above.

S10.3 Constraint counts

For Q8 the 192 factor and public limbs contribute $192 \cdot 65 = 12480$ rows: 64 bit equations and one reconstruction equation per limb. The 4096 partial products contribute 4096 rows. The 127 intermediate carries contribute $127 \cdot 71 = 9017$ rows: 70 bit equations and one reconstruction each. Add 128 column equations, two zero-endpoint equations and two factor lower-bound equations, for 25725 rows in total. Auxiliary signals are supplied explicitly. The signal count is $192 \cdot 65 + 4096 + 129 + 127 \cdot 70 + 2 = 25597$. For Q7, three 64-bit decompositions, one product and four inverse equations give $195 + 1 + 4 = 200$ rows. The 65-row range gadget and one-row exclusion gadget complete the four families.

Generated results

N	Multiplies	Adds	Rotations	Depth	Cost (s)
32768	4757	4754	433	3	29.299516
65536	13313	13310	570	3	78.083040

For $N = 32768$, stages are $[7, 7, 1]$, baby sizes $[2048, 32, 256]$, and the dense baseline is 185.962068 seconds (6.347 times the selected cost). The 13 target cases have maximum normalised error 2×10^{-15} . For $N = 65536$, stages are $[8, 7, 1]$, baby sizes $[4096, 32, 256]$, and the dense baseline is 370.374740 seconds (4.743 times the selected cost). The 13 target cases have maximum normalised error 2.13×10^{-15} .

n	1	3	5	7	9	11	13	15
Gates	0	1	7	30	88	205	411	742

Circuit	Compiled rows	Signals
Q5	65	65
Q6	1	2
Q7	200	199
Q8	25725	25597

The compiled audit records 66 passing checks. The challenge and affected Boolean regression suites collect 63 and 525 tests respectively; all pass. The modular enumeration counts for primes 3, 5, 7, 11, 13, 19 are 9, 25, 1, 1, 73, 1. Tables are generated from the recorded verification results; individual records retain the producing commands and source hashes. The supplementary index check reconstructs 2730 majority circuit states. The incorrect nine-input circuit disagrees with majority on 54 of 512 states, despite having the same functional inputs and the same number of active indices.

S10.4 Reproduction and scope of the evidence

Reproduction proceeds in four parts: regenerate the discrete constructions and compare their complete or symbolic outputs; emit and audit the four quadratic systems; execute both selected Fourier circuits and compare their results; and regenerate the tables and documents from the resulting records. The accompanying reproduction guide supplies executable commands and dependency pins separately from the mathematical exposition.

The original verification record contains 63 challenge tests and 525 affected Boolean regression tests. It also records 66 compiled arithmetic checks and 13 plaintext vectors at each target Fourier size. Missing tools, skipped required tests and unresolved outcomes are failures of full verification. The new index-deconvolution examples are a separate, bounded manuscript check, described in Section S4. The original numerical and compiled results are retained; rebuilding the documents does not rerun those experiments. Records retain the producing commands, source hashes, numerical tolerances, input sizes, seeds, elapsed times and observed memory. Document validation checks successful compilation, resolved references and page bounds. It does not replace the mathematical or computational checks.

What has not been measured. No complete 2025-input circuit has been materialised, no ciphertext Fourier timing has been taken, and no zero-knowledge proof has been generated. The arithmetic results establish properties of the compiled constraint systems and supplied witnesses. They are not a benchmark of proving time, proof size or a deployed cryptographic protocol.

S11 General capacities and developmental scope of index deconvolution

The index-deconvolution method is the Boolean-network component of the broader CausalBool programme. Its conceptual lineage is the algorithmic information-dynamics and programmability framework developed for analysing how systems respond to perturbations, how short rules can describe their behaviour, and how such descriptions can support explanations and predictions [2, 3]. The present method is a specialisation of that perspective to an exact, finite Boolean repertoire. It should therefore be read as strongly aligned with the spirit of Kolmogorov–Chaitin complexity, while remaining distinct from a computation of Kolmogorov complexity itself.

S11.1 The capacity map

For a synchronous Boolean network, the forward CausalBool representation maps a connectivity matrix and node rules to an ordered repertoire. The inverse method starts with that repertoire and works one output column at a time. For column k , a coordinate i is functional exactly when there is a state x such that

$$O_k[x] \neq O_k[x \oplus 2^i]. \quad (\text{S1})$$

The resulting set is the set of connected inputs that the behaviour itself determines. Restricting the column to those inputs gives a reduced truth table. Matching that table against the canonical gate family, or retaining it as an explicit look-up table when no named gate matches, produces a generative description that can be replayed independently.

The main capacities and their exact boundaries are summarised below. The word “exact” always refers to the stated finite model and its supplied repertoire; it does not mean that an arbitrary hidden implementation is uniquely identifiable.

Figure 1 displays these operations in the same network–matrix–pattern style as the worked majority figures. It uses the five-input circuit already checked in this response: the complete repertoire is the input to the inverse, paired perturbations expose functional inputs, the reduced behaviour is named or retained exactly, and the schema view gives a lossless expansion into output indices. The final panel closes the loop by replaying the recovered description and answering a deterministic query.

S11.2 Compression and the Kolmogorov perspective

The schema representation makes the algorithmic-information connection concrete. A schema fixes some coordinates and leaves others as don’t-cares. Its fixed coordinates determine a decimal anchor P ; the fillings of its own don’t-care positions form the sumando family S . Across a decimal family L of anchors, the decoder expands the representation by

$$\text{Dec}(L, S) = \{\ell + s : \ell \in L, s \in S_\ell\}. \quad (\text{S2})$$

The expansion is checked against the original output-one index set, so this is lossless representation rather than a probabilistic approximation. When a function has repeated structure or overlapping regularities, the schema description can be substantially more compact than listing every active index. For functions such as XOR, however, the same representation can offer little or no compression. Compression is therefore a capacity to be measured for a specified function, not a property to be assumed in advance.

Kolmogorov complexity $K(x)$ is the length of the shortest halting program that produces x and is uncomputable in general. A fixed index-set encoding therefore supplies a computable description length, and hence an upper-bound candidate relative to that encoding, rather than $K(x)$ itself.

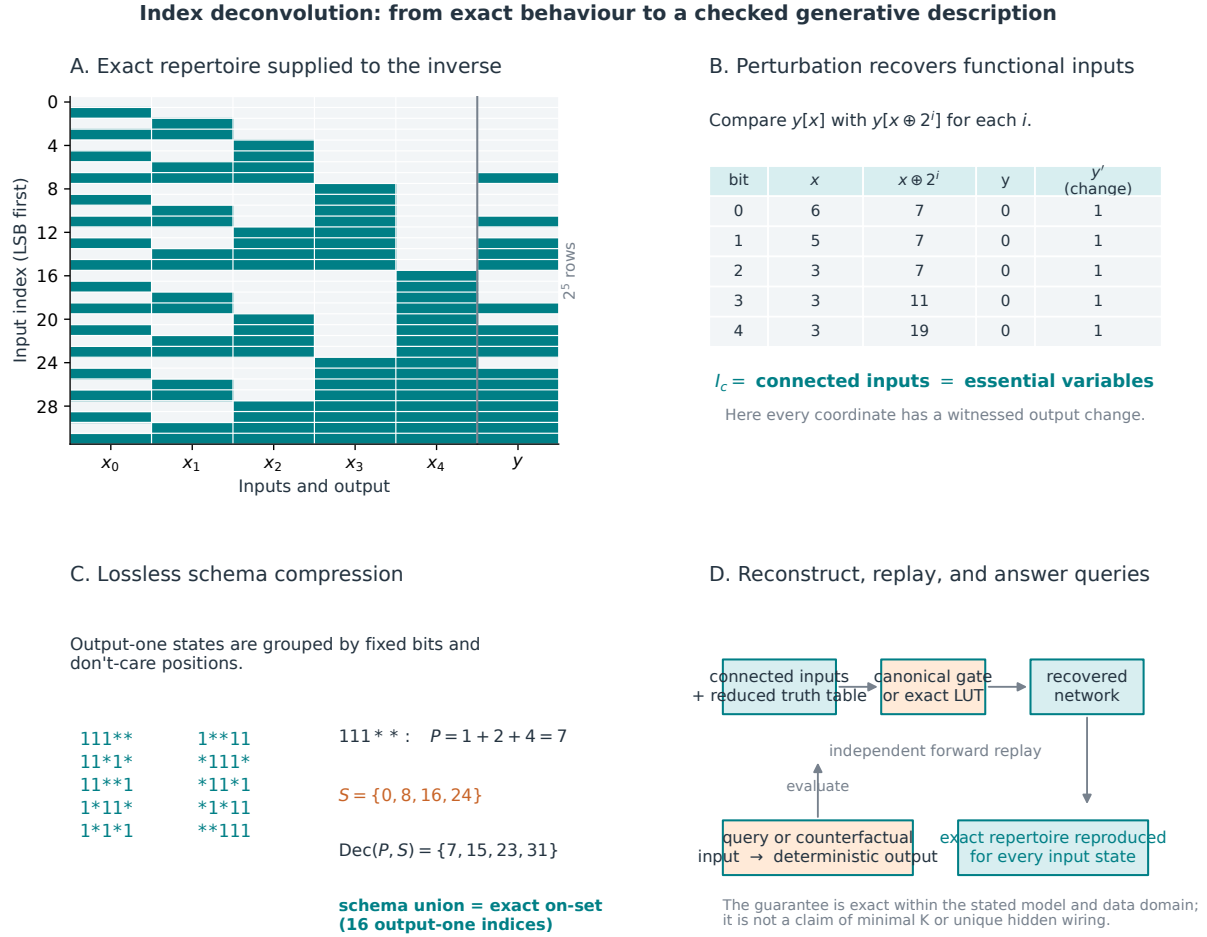


Figure 1: Capacities of index deconvolution. The actual five-input majority example is followed from its complete repertoire through paired single-bit perturbations, recovery of connected inputs, schema-based description and independent replay. The schema example has decimal anchor $P = 7$ and sumandos $S = \{0, 8, 16, 24\}$, whose exact expansion is $\{7, 15, 23, 31\}$. The diagram is a visual summary of finite exact operations, not a claim that the method computes minimal Kolmogorov complexity or identifies hidden wiring uniquely.

Capacity	What the method does	Boundary of the claim
Exact representation	Represents the complete Boolean behaviour as an ordered index/output repertoire and can expand a compact description back to that repertoire.	Requires a deterministic, correctly ordered finite repertoire.
Functional deconvolution	Detects connected inputs by paired single-bit perturbations and reduces each output column to its functional truth table.	Recovers functional dependence, not edges that the function never uses.
Gate identification	Matches a reduced truth table to canonical gates and reports the full equivalence class of matching names.	Different internal circuits or gate names can realise the same function.
Lossless compression	Groups output-one indices into schemata using decimal anchors, decimal families and sumandos; schema expansion reproduces the on-set exactly.	The description is a computable representation and need not be shorter for every function.
Perturbation analysis	Supplies witnesses for whether changing one input can change an output, giving deterministic functional-sensitivity and counterfactual checks.	This is model-based intervention analysis, not a general causal claim from observational data.
Prediction by replay	Evaluates a recovered function for a queried input and checks the prediction by independent forward replay.	It is exact within the model/data domain, not an out-of-sample forecast for noisy or incomplete data.

Table 1: Capacities and boundaries of the index-deconvolution method. The method separates what is recoverable from a complete repertoire from what remains non-identifiable.

The method does not establish minimality across all programming languages, coding schemes or representations. Its algorithmic character comes from describing an individual deterministic object by a generative rule and testing whether that rule reproduces the object exactly.

S11.3 Prediction, explanation and scale

Once the functional inputs and reduced truth table have been recovered, a new input can be evaluated without consulting the original hidden circuit. Schema membership gives the same answer for an input in the represented domain, while a paired state $x \oplus 2^i$ supplies a direct witness for whether coordinate i can affect the output. In this limited but precise sense, the method supports prediction and explanation together: the predicted output is accompanied by the recovered rule, the inputs on which it depends, and the exact repertoire-replay check. This is the model-driven explanatory role that motivates the perturbation framework in the earlier work [2, 3].

The computational boundary is equally important. Given the full $2^n \times n$ repertoire, exhaustive paired checks for all columns take $O(n^2 2^n)$ time, but obtaining and storing that repertoire already costs an exponential number of states. Exact inversion is therefore practical for small networks and for larger systems only when an additional structural condition, such as bounded locality, makes the relevant observations effectively complete. Partial, noisy or non-recurrent observations require a different approximate problem and are not silently treated as exact deconvolution.

The method remains in development. The current implementation establishes finite exact reconstruction and lossless schema expansion for the checked Boolean cases; it does not yet provide a universal minimal description, representation-invariant complexity value, unique recovery of internal gate wiring, or a general out-of-sample predictor. Larger-network trajectories, partial

observations, richer dynamical systems and principled comparisons of description lengths remain development directions. These qualifications are part of the method’s scientific specification rather than caveats added after the result.

References

- [1] 0xPARC. *Beyond Primitive Computing: Puzzles and Connections*, Field Extension. <https://fielddextension.org/3b1b/>.
- [2] A. Hernández-Espinosa, H. Zenil, N. A. Kiani and J. Tegnér, *Estimations of Integrated Information Based on Algorithmic Complexity and Dynamic Querying*, arXiv:1904.10393 [q-bio.NC] (2019). <https://arxiv.org/abs/1904.10393>.
- [3] A. Hernández-Espinosa, H. Zenil, N. A. Kiani and J. Tegnér, “Estimations of Integrated Information Based on Algorithmic Complexity and Dynamic Querying,” in *Handbook of Unconventional Computing*, Vol. 1: Theory, ed. A. Adamatzky (World Scientific, 2021), Chap. 5, pp. 171–220. https://doi.org/10.1142/9789811235726_0005.
- [4] G. Cohen, I. B. Damgård, Y. Ishai, J. Kölker, P. B. Miltersen, R. Raz, and R. D. Rothblum. *Efficient Multiparty Protocols via Log-Depth Threshold Formulae*, 2013. https://www.wisdom.weizmann.ac.il/~ranraz/publications/Pmajority_mpc-1.pdf.
- [5] J. H. Cheon, K. Han, and M. Hhan. *Faster Homomorphic Discrete Fourier Transforms and Improved FHE Bootstrapping*. Cryptology ePrint 2018/1073. <https://eprint.iacr.org/2018/1073>.
- [6] iden3. *Circom 2.2.3 and snarkjs 0.7.6*. <https://docs.circom.io/>; <https://www.npmjs.com/package/snarkjs/v/0.7.6>.